

Few-shot Generation with LLMs

1. Background

In previous assignments (e.g., HW3), you have gained a deeper understanding of the Transformer architecture by implementing the core components of a decoder. That paradigm, focused on *training* or *fine-tuning* a model's weights, is powerful but resource-intensive. This homework shifts our focus to a different paradigm that has emerged with Large Language Models (LLMs). LLMs are pretrained on massive-scale text corpora and this extensive pretraining process imbues them with a wide range of general-purpose capabilities and world knowledge. These capabilities can be invoked at inference time without any gradient updates to the model's parameters, a phenomenon known as In-Context Learning (ICL). Instead of re-training the model, we "program" it using natural language prompts. Our focus shifts from *architecture engineering* to Prompt Engineering (how we frame the input to elicit the desired behavior) and Inference-time Strategies (how we generate and select the final output).

In this assignment, you will implement and compare two fundamental prompting techniques, Zero-shot and Few-shot, and an inference time scaling strategy, Majority Voting, to enhance performance. You are required to implement the following functionalities, which will form the basis of your experimental report.

1. Implement Few-shot Prompting

- You must implement and compare **Zero-shot** and **Few-shot** prompting.
- Given a test question x , a **Zero-shot** prompt P_{zero} is typically a direct formatting of the question (e.g., `Q: [x] A:`). The model is expected to generate the answer y by computing $P(y|P_{\text{zero}})$.
- A **Few-shot** prompt P_{few} provides k in-context examples $\{(x_1, y_1), \dots, (x_k, y_k)\}$. The prompt is constructed by concatenating these examples with the new question:
 $P_{\text{few}} = \text{Format}(\langle x_1, y_1 \rangle, \dots, \langle x_k, y_k \rangle, x)$. The model then computes $P(y|P_{\text{few}})$.
- Your task is to build a template that can format k examples. On the `gsm8k` and `math500` datasets, compare the performance differences between Zero-shot and Few-shot (e.g., $k = 1, k = 3, k = 5$).

2. Implement Majority Voting

- You will implement a simplified version of "Self-Consistency," referred to as **Majority Voting**.
- This strategy relies on Temperature Sampling. Instead of deterministic greedy decoding, you sample from the model's output distribution, which is "flattened" by a temperature $T > 0$. Given the logits z for the next token, the probability of sampling token w_i is:

$$P_T(w_i | \text{context}) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

- You must generate N independent full responses $\{y^{(1)}, \dots, y^{(N)}\}$ by sampling with $T > 0$ (e.g., $N = 5, N = 10$). After extracting the final numerical answer $a^{(i)}$ from each response $y^{(i)}$, the final output a_{final} is the one with the highest frequency (the mode):

$$a_{\text{final}} = \underset{a}{\operatorname{argmax}} \left| \{i \mid a^{(i)} = a, 1 \leq i \leq N\} \right|$$

- On the `math500` dataset, observe and report the change in performance compared to a single greedy decoding pass (which is equivalent to $N = 1$ and $T \rightarrow 0$).

3. Analysis and Reflection

- **The Importance of Few-shot:** Reflect on when Few-shot prompting is most critical. Provide a case study (e.g., a non-standard task like: `count how many 'r's in strawberry`). Analyze why Zero-shot might fail in this case, whereas Few-shot helps the model "understand" the task format.
- **Effectiveness of Majority Voting:** Analyze the conditions under which Majority Voting works. You should try to analyze the performance improvement stratified by the **difficulty** of the `math500` problems (you may define your own criteria for simple, medium, and hard).

2. Python Files Description

In this project, we encapsulate math problems as `Math` class in `task.py`, in which the `test` method is for you to test the effect of your prompt/inference-time scaling. Each prompting method is defined as a function in `prompt.py`:

- `zero_shot`: Return zero-shot prompt;
- `few_shot`: Return prompt with several shots. You can change the number of shots via `shot` parameter.

Similarly, inference-time scaling methods are also defined as functions in `scale.py`:

- `majority_voting`: Return the final result after the voting process;
- Other inference-time scaling methods.

The entire process is conducted in `main.py`. You should run `main.py` with parameters below:

- `base_url`: The address of the API endpoint;
- `api_key`: The key for calling the API;
- `model`: The model being called;
- `dataset`: The currently specified task, which can be one of math-500, gsm8k, or others;
- `task`: The prompt method setting for the current task, which can be zero-shot, few-shot, or scaling.

All files included in the codes:

- `main.py`: The main script for running the whole program.
- `prompt.py`: You should implement `zero_shot` and `few_shot`.
- `scale.py`: You should implement `majority_voting`, and other inference-time scaling methods for bonus.
- `task.py`: Implementation of math problem task.
- `api.py`: Include a function for communicating with models via API.
- `data`: Include two datasets for math ability: Math500 and GSM8k.
- `utils`: Include tools that will be used in task-examine process. You should **NEVER** change codes in `utils` directory.

We also provide a reference script in `test.sh`. You can modify the parameters in this script and run test with it.

We highly recommend using `vllm` (<https://github.com/vllm-project/vllm>) to deploy test models locally and finish the homework, but we also provide API for Qwen2.5-7B-Instruct. If you choose to deploy models locally, you need to briefly illustrate your implement in the report, and these codes do not require automated Code Checking.

When using provided API, we highly recommend firstly using several questions to debug and then testing on the 500 question set.

3. Requirements

- `python >= 3.8`
- `PyTorch >= 1.10`
- `transformers`
- `datasets`
- `numpy`
- `OpenAI`

4. Dataset & Models & Metric

Dataset

- **GSM8K**: A dataset of ~8.5K high-quality, linguistically diverse grade school math word problems. In this experiment, you only need to select the first 500 entries.
- **Math500**: A dataset containing 500 math problems (assumed to be a subset or custom collection based on the [MATH dataset](#)).

Target Models

- [Qwen2.5-7B-Instruct](#)

Metric

- **Accuracy**: For these mathematical reasoning tasks, we evaluate success based on whether the final extracted answer is identical to the ground-truth answer.

5. Report

Your report should include the following experiments and analyses:

1. Experiment 1 (Few-shot vs. Zero-shot) : For this experiment, you are required to conduct a comprehensive comparison of Zero-shot versus k -shot prompting ($k > 0$). Your analysis must present clear tables or bar charts illustrating the accuracy results on both the gsm8k (first 500 entries) and math500 datasets. You are encouraged to explore the relationship between the quantity of demonstrations and the resulting accuracy.

2. Experiment 2 (Few-shot Case Study): In this section, you will provide a qualitative Few-shot Case Study based on your reflections from Experiment 1. You need to analyze whether the improvement on gsm8k is significant when using few-shot prompts, and find another scenario where few-shot helps more, such as the "count 'r's in strawberry" task or a similar non-standard task. You are expected to show the exact Zero-shot and Few-shot prompts you constructed, the model's corresponding outputs for each prompt, and a thorough analysis explaining the differences in the results.
3. Experiment 3 (Majority Voting Performance) : For Experiment 3, you should evaluate the performance of Majority Voting on the math500 dataset. This involves a direct accuracy comparison between the Baseline (defined as greedy decoding with $N = 1$) and the Majority Voting strategy (where $N > 1$ and $T > 0$). Your report should also include a specific analysis of the impact of N , examining how changing the number of generated samples influences the final accuracy of the model.
4. Experiment 4 (Majority Voting In-depth Analysis): This experiment requires an In-depth Analysis of Majority Voting's effectiveness. Drawing on your work from Experiment 3, you should first stratify the math500 dataset by difficulty (using criteria defined by yourself). You will then analyze whether the performance gained from Majority Voting is consistent across all difficulty levels. Your findings should be supported by an explanation for why this strategy's effectiveness might or might not vary based on problem complexity.

6. Bonus (≤ 2)

1. Impact of Model Scale: For this bonus, you will investigate the Impact of Model Scale on few-shot learning. You are required to repeat Experiment 1 (Few-shot vs. Zero-shot) using various models from the Qwen2 series (e.g., 0.5B, 1.5B, 3B, 7B, 14B). You should then observe and provide a detailed analysis of how model scale affects both the overall performance and the sensitivity of the models to few-shot prompting.
2. MCTS Implementation: This bonus challenges you to implement an advanced inference-time strategy, specifically Monte Carlo Tree Search (MCTS) or another comparable scaling method. After implementing your chosen strategy, you should design and run experiments to compare its performance directly against the Majority Voting baseline established in Experiment 3.
3. Multimodal Reasoning Research: This bonus is a research task focusing on the spatial reasoning capabilities of modern multimodal models. You could investigate how these models handle tasks involving relative object positions or geometric relations in images. Your submission should provide two to three detailed case studies, including both successful and failed examples, along with your analysis of the observed phenomena.
4. Architectural Comparison: This task involves a research-based Architectural Comparison. You are required to investigate the Decoder-only architecture of modern LLMs (e.g., Qwen2, Llama) and compare it to the Transformer decoder you implemented in HW3. Your analysis should highlight key structural differences and variations in component design.

7. Code Checking

We introduce a code checking tool this year to avoid plagiarism. You **MUST** submit a file named `summary.txt` along with your code, which contains what you modified and referred to. You should follow the instructions below to generate the file:

1. Fill the codes. Notice you should only modify the codes between `# TODO START` and `# TODO END`, the other changes should be explained in README.txt. **DO NOT** change or remove the lines start with `# TODO`.
2. Add references if you use or refer to a online code, or discuss with your classmates. You should add a comment line just after `# TODO START` in the following formats:

1. If you use a code online: `# Reference: https://github.com/xxxxx`
2. If you discuss with your classmates: `# Reference: Name: Xiao Ming Student ID: 2018xxxxxx`

You can add multiple references if needed.

3. Here is an example of your submitted code:

```
def forward(self, input):
    # TODO START
    # Reference: https://github.com/xxxxx
    # Reference: Name: Xiao Ming Student ID: 2018xxxxxx
    your codes...
    # TODO END
```

4. At last, run python `./code_analyze/analyze.py`, the result will be generated at `./code_analyze/summary.txt`. Open it and check if it is reasonable. A possible code checking result can be:

```
#####
# Filled Code
#####
# ..\codes\layers.py:1
#   # Reference: https://github.com/xxxxx
#   # Reference: Name: Xiao Ming Student ID: 2018xxxxxx
#   your codes...
#####
# References
#####
# https://github.com/xxxxx
# Name: Xiao Ming Student ID: 2018xxxxxx
#####
# Other Modifications
#####
# _codes\layers.py -> ..\codes\layers.py
# 8 - self._saved_tensor = None
# 8 + self._saved_tensor = None # add some thing
```

8. Submission Guideline

You need to submit a `.zip` file named after your student number, organized as below:

1. **Report.pdf**: Your well-formatted report detailing your implementation, experimental setup, results, and analysis.
2. **codes/**: All your source code `.py` files.

3. **README.txt:** A brief description of your code structure and clear instructions on how to reproduce your results (e.g., what scripts to run). DO NOT include model weights, raw data, or large cache files.

9. API Key

Considering that some students may not have the conditions for local model deployment, we have prepared a separate API key for each student for model inference. When in actual use, you can simply pass the API key as a parameter. The API key can be obtained by running the following command:

```
curl "http://115.182.62.174:18887/query?student_id=<your_student_id>"
```

Then you will get your API key contained in a json. Each API key has a quota of \$10, which is enough to finish the homework. Please use the quota as needed and avoid any waste. If you have used up the quota of your API key, please contact the TA and explain your detailed usage.

10. Deadline & Contact

- **Deadline:** 11.30
- **TA contact:** 杨峻骁、刘洲甫